

CCNA 200-301 V2.0

33

A Troubleshooting Method for the 28 Per Cent

Part VI — Sitting the Exam

EXAM TOPICS

2.4

LECTURER

Dr. Mustafa Hameed



THE ISLAMIA UNIVERSITY OF BAHAWALPUR

Department of Information Technology

Where this chapter sits

Part I	Foundations	chapters 1–7
Part II	Switching and Network Access	chapters 8–14
Part III	IP Routing	chapters 15–19
Part IV	Network Services and Security	chapters 20–26
Part V	Wireless, Virtualisation and Operations	chapters 27–32
Part VI	Sitting the Exam	chapters 33–34

What you will be able to do

- **Apply** a seven-step cycle to any fault, rather than guessing in a familiar order.
- **Choose** between the five structured methods and justify the choice from the symptom.
- **Scope** a fault before touching anything, using four questions that routinely halve the search.
- **Work** evidence-first: record before you change, change one thing, and undo what did not help.

Before we start

Everything. This chapter is the method that the rest of the book supplies the knowledge for. Chapter 6 for the diagnostic tools and Chapter 14 for layer 2 in particular; the symptom index in Section `sec:symptomindex` points at all the others.

Vocabulary for this chapter

scoping

top-down

bottom-up

divide and conquer

follow the path

compare working to broken

hypothesis

blast radius

handover

Each is defined where it first matters — not here.

What topic 2.4 actually says

This is what the blueprint actually asks for

Troubleshoot basic Layer 2/Layer 3 connectivity and device operations using show commands (including show logs), ping, extended ping, trace route, and packet capture output.

Every tool in that list has already been taught — Chapter 6 for ping, extended ping, traceroute and reading captures; Chapter 30 for `show logging`; Chapter 14 for layer 2. This chapter deliberately does not repeat them.

What it adds is the thing the tools cannot supply: **an order to use them in**. Eight of the twenty-nine topics begin *Troubleshoot* or *Diagnose*, which is 28% of the blueprint, and in every one of them the examiner already knows you can type `show ip interface brief`. What is being tested is whether you can decide to.

The failure mode to design against

The characteristic mistake is **finding a fault and stopping**. There is a misconfiguration on the path, you fix it, and the symptom persists — because it was not *the* fault, or not the only one.

A method protects against this by making you state what you expect the fix to do *before* you apply it. If the prediction fails, you have learned something; if you had no prediction, you have only changed something.

The cycle



The cycle



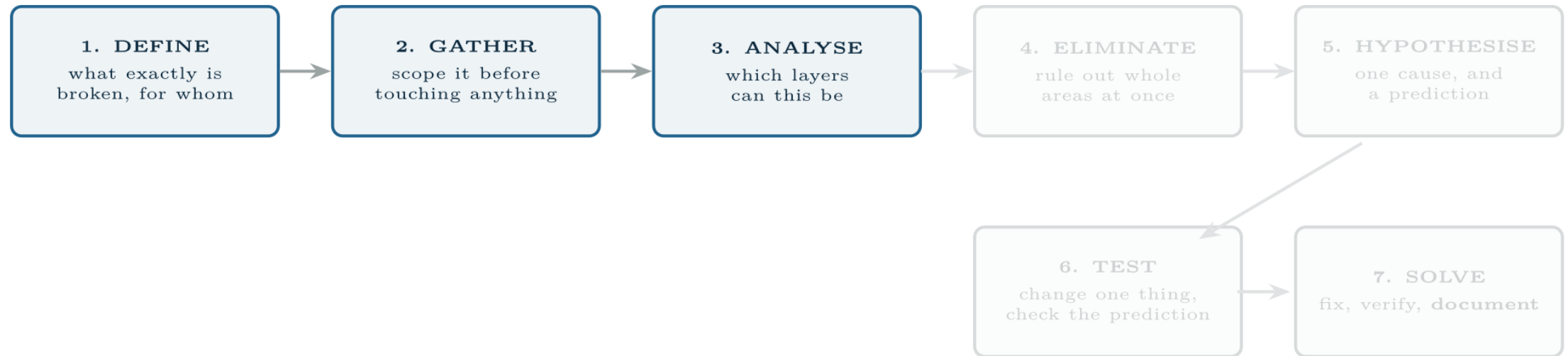
“The network is down” is not a fault. Who, what, since when, and what still works?
Seven steps — and the three that get skipped under pressure are 1, 2 and 5.

The cycle



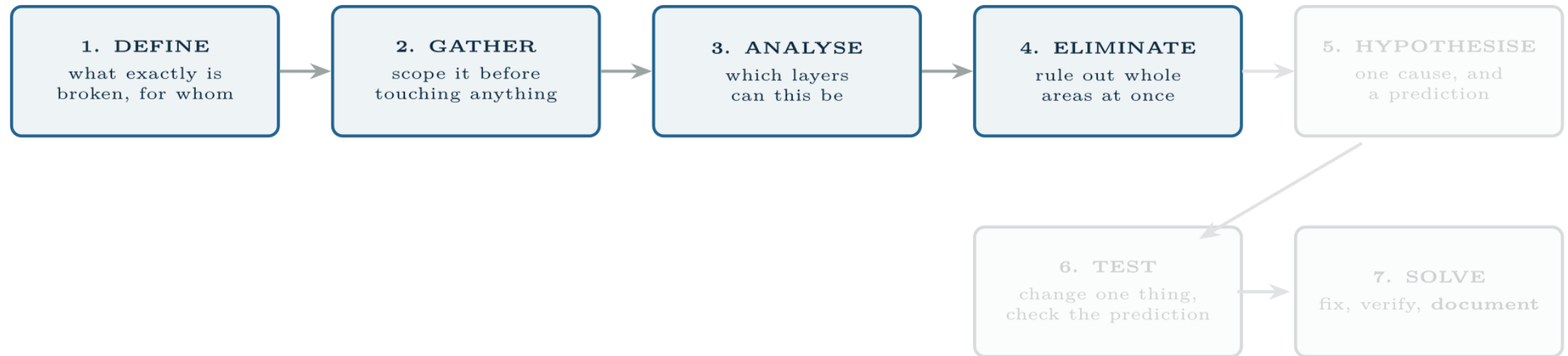
Scope it *before* touching anything. Once you have changed something you can no longer tell what the fault was.

The cycle



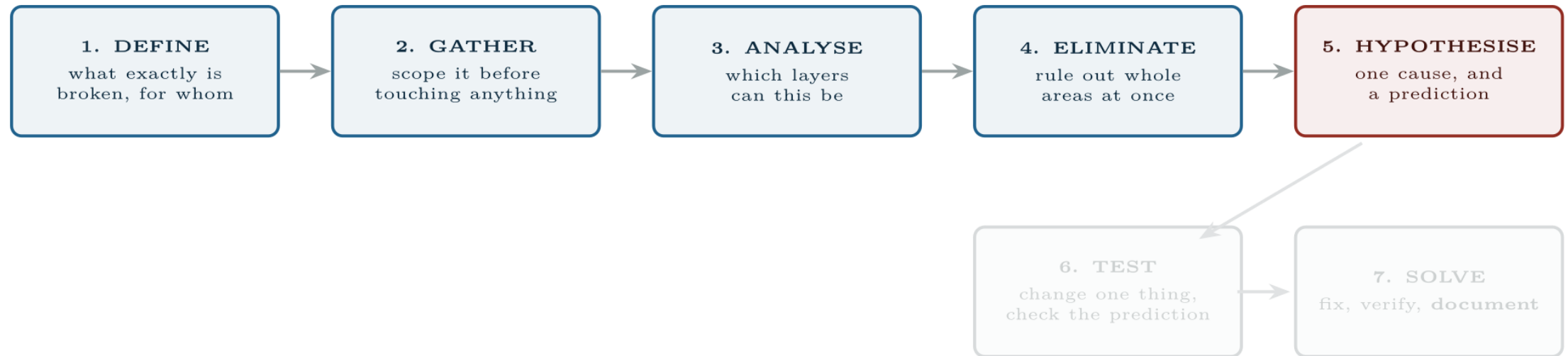
Now decide which layers this can possibly be. Most of the stack is usually innocent, and saying so out loud is what stops the guessing.

The cycle



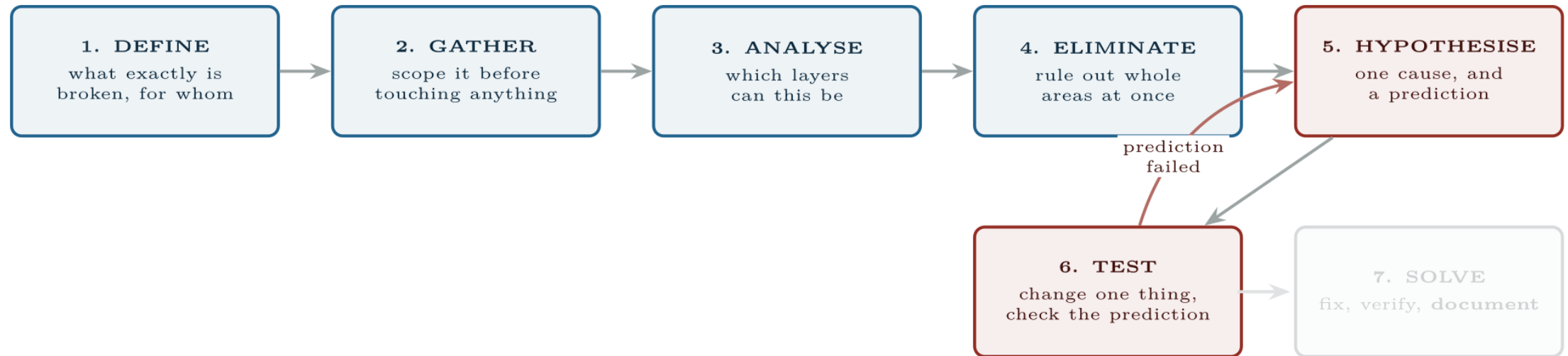
Rule out whole areas at once. One ping to the gateway clears layers 1 to 3 on this segment — that is four steps of a bottom-up search in a single command.

The cycle



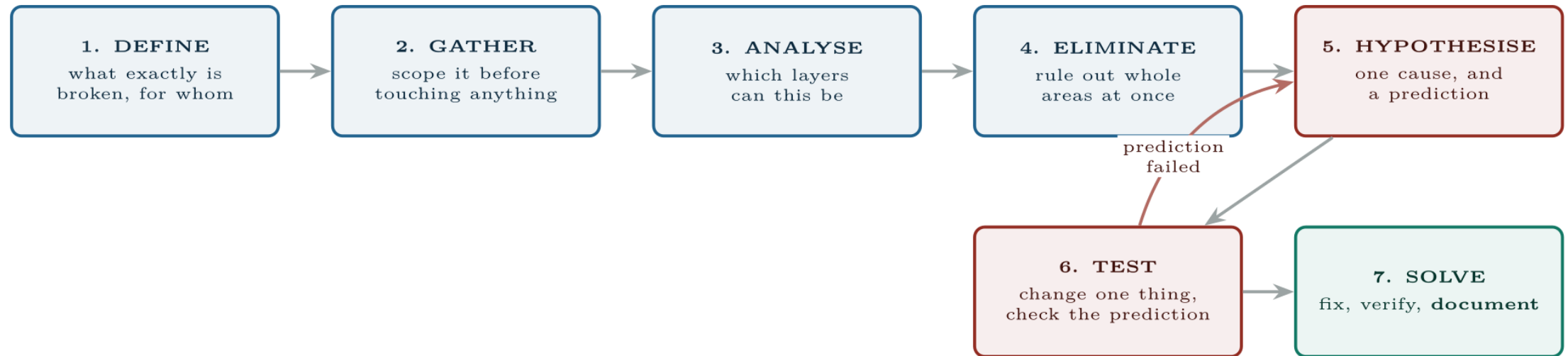
One cause, and a prediction — “if I am right, this command will show that”. A guess with no prediction cannot be wrong, which is exactly what makes it useless.

The cycle



Change one thing, and check the prediction. If it failed the hypothesis is discarded — not patched — and you go straight back to 5. That arc is the method; everything above it is preparation.

The cycle



And only then fix it. Document is part of step 7, not an optional extra: the next person to meet this fault is usually you, six months later.

Step 5 is where amateurs and professionals separate

“I think it is the trunk allowed list” is not a hypothesis. **“I think VLAN 30 is missing from the allowed list on the uplink, in which case `show interfaces trunk` will not list it, and VLANs 10 and 20 will be unaffected”** is one — because it can be wrong, and you will know immediately.

Say what you expect to see before you run the command. It takes three seconds and it converts every check into information whichever way it goes.

Compare-to-working is under-used and enormously effective

Twenty-four ports configured the same way, one of them broken. Do not reason about it — `show running-config interface` on both and read the difference.

The same applies to a device: if three hypervisor hosts work and the fourth does not (Chapter 29), the fault is in what is *not* identical, and that is a much smaller search space than “what could be wrong with a hypervisor uplink”.

Whenever a working comparison exists, use it first. It converts an open-ended diagnosis into a diff.

“What changed?” is the highest-yield question in operations

A network that worked on Monday and fails on Tuesday was changed. The change may be undocumented, may have been made by someone else, may not look related, and the person who made it may sincerely believe it could not have caused this.

Ask anyway, ask broadly, and check the evidence rather than the recollection: `show logging` for %SYS-5-CONFIG_I messages (Chapter 30), `show archive config differences`, and the change record.

The answer is often “nothing”, and it is often wrong.

The exception, and know that you are taking it

Under a major outage there is pressure to restore service first and understand later. That is a legitimate decision — but take it deliberately, say out loud that you are taking it, and capture what you can as you go. The failure is not choosing speed; it is choosing speed by accident and having no idea afterwards what you did.

Handing it over



Common pitfalls

- **Starting to type before scoping.** Four questions first.
- **Checking what broke last time.** Familiarity is not evidence.
- **No prediction before a change.** You learn nothing from the result.
- **Two changes at once.** Now you do not know which worked.
- **Leaving failed changes in place.** They become the next fault.
- **Fixing the first fault found and declaring victory** without re-testing the original symptom.
- **Ignoring an available working comparison.**

Common pitfalls (continued)

- **Believing “nothing changed”.** Check the logs.
- **Not writing it down.** You will meet this fault again.

Two faults at once, and fixing the first one changes nothing.

Users on the second floor report that the file server is unreachable. You find that the uplink from SW2 has VLAN 20 missing from its trunk allowed list, which would certainly do it. You add it, confirm `show interfaces trunk` now lists VLAN 20 — and the users report no improvement at all.

The instinct is to assume the fix did not take, and to re-check the trunk. Resist it: you verified it, and it is correct. A correct fix that does not resolve the symptom means **your hypothesis explained a real fault, but not this one.**

Go back to scoping, which is where the answer was all along.

Two faults at once, and fixing the first one changes nothing.

What would you check first — and what do you expect to see?

show ip route 10.20.0.0

```
R1#
```

```
% Network not in table
```

What is the fault — and what does this output rule out?

Why it is doing that

Diagnosis. The server's subnet is not in the routing table. Its route was lost overnight — and *that* is why nobody can reach it, from any floor.

The missing VLAN on SW2's trunk was a second, genuine, unrelated fault. It had been there for weeks without being noticed, because nothing in VLAN 20 on the second floor had needed the uplink until the server was investigated. Finding it was useful. Believing it was the cause was the error.

And the lesson

The lesson, and it is the one this chapter exists for. Two things went wrong here, and only one of them was reported:

Faults you did not create

Platform note. The value here is entirely in *not* knowing the fault, so this lab must be done in pairs or by exchanging saved files. Working alone on a fault you injected yourself teaches nothing.

Topology. Any multi-VLAN, multi-router topology from Parts II and III that both of you have working.

1. Each of you takes a working copy and injects **three** faults from different layers — one physical or layer 2, one layer 3, one a service (DHCP, DNS, NAT or an ACL). Record them privately. At least one should be silent, producing no log message.
2. Exchange files. Before running a single command, write down your scoping questions and what each possible answer would eliminate.
3. Diagnose. For every check, write the prediction *before* you run it, and whether it held. Keep this record — it is the deliverable.
4. Fix one fault at a time and re-test the original symptom after each. Note any case where a correct fix did not change the symptom, and what you concluded.

Faults you did not create (continued)

5. Time yourself. Compare against your partner on the same set of faults.
6. Write the handover you would give if you ran out of time halfway.
7. **Round 2.** Inject three more, but this time include **two faults on the same path** so that fixing one changes nothing. This is the Break & Fix above, and it is the most valuable version of this exercise.
8. **Round 3.** Inject one fault into *one* of twenty-four identically configured ports. Time how long it takes with reasoning versus with a configuration diff.
9. **Extension.** Build a one-page fault-finding card for your own use: the four scoping questions, the five methods, and the symptom index trimmed to what you personally forget.

CHECKPOINT 1 of 6

Give the seven steps of the cycle and say which two form a loop.

Discuss · then advance

Checkpoint 1 of 6

Give the seven steps of the cycle and say which two form a loop.

Define, gather, analyse, eliminate, hypothesise, test, solve. Steps **5 and 6** form the loop — a hypothesis whose prediction fails is discarded and replaced, not patched.

CHECKPOINT 2 of 6

Name the five structured methods and give the symptom that selects each.

Discuss · then advance

Checkpoint 2 of 6

Name the five structured methods and give the symptom that selects each.

Bottom-up for a suspected physical or hardware fault, or when nothing works. Top-down when one application fails and everything else is fine. Divide and conquer as the default when you have no strong prior. Follow the path when two specific points cannot reach each other. Compare working to broken whenever an identical working case exists.

CHECKPOINT 3 of 6

What are the four scoping questions, and what does each eliminate?

Discuss · then advance

Checkpoint 3 of 6

What are the four scoping questions, and what does each eliminate?

One user or many? — sets the whole search space. *Has it ever worked?* — separates a build fault from a change fault. *What changed?* — finds the cause of most faults in previously working networks. *Does it fail by name but work by address?* — answers “is this DNS” in one test.

CHECKPOINT 4 of 6

Why must you state a prediction before making a change?

Discuss · then advance

Checkpoint 4 of 6

Why must you state a prediction before making a change?

Because it converts a check into information whichever way it goes. Without a prediction, a fix that does not resolve the symptom reads as “the fix did not work” rather than “the hypothesis was wrong”, and you re-check something already correct.

CHECKPOINT 5 of 6

A correct fix does not resolve the reported symptom. What does this tell you, and what should you do next?

Discuss · then advance

Checkpoint 5 of 6

A correct fix does not resolve the reported symptom. What does this tell you, and what should you do next?

Either the hypothesis was wrong, or there is more than one fault. Go back to scoping rather than re-checking the thing you just verified — and keep the fix if it corrected a genuine defect, but record it separately.

CHECKPOINT 6 of 6

What are the four things a handover must contain?

Discuss · then advance

Checkpoint 6 of 6

What are the four things a handover must contain?

The precise symptom and who is affected; what has been ruled out and the evidence for it; what has been changed, including anything not yet undone; and the current hypothesis with the next test.

What to take away

- Define, gather, analyse, eliminate, hypothesise, test, solve — and loop between hypothesis and test until the prediction holds.
- A hypothesis must predict something that could be wrong. State it before you run the command.
- Five methods: bottom-up for physical, top-down for one application, divide and conquer as the default, follow the path between two points, and compare-to-working whenever an identical working case exists.
- Scope first: one user or many, has it ever worked, what changed, does it fail by name but work by address.

What to take away (continued)

- “What changed” is the highest-yield question, and “nothing” is often wrong. Check [show logging](#) and the archive diff.
- Record before changing, change one thing, undo what did not help.
- A correct fix that does not resolve the symptom means the hypothesis was wrong — or that there is more than one fault.
- A handover states the symptom, what has been ruled out and how, what was changed, and the next test.

Where we got to

- Define, gather, analyse, eliminate, hypothesise, test, solve — and loop between hypothesis and test until the prediction holds.
- A hypothesis must predict something that could be wrong. State it before you run the command.
- Five methods: bottom-up for physical, top-down for one application, divide and conquer as the default, follow the path between two points, and compare-to-working whenever an identical working case exists.
- Scope first: one user or many, has it ever worked, what changed, does it fail by name but work by address.

NEXT

Chapter 34 — Exam Strategy